# East West University

**Course Code & Name:** CSE 246 (Algorithms)
**Course Instructor:** Sanzana Karim Lora
**Course Section:** 10
**Academic Session:** Summer 2025

# Project Report

## Group No.- 04

**Group Members:**

| Name | Student ID |
|---|---|
| **MD Abir Rahman** | 2023-3-60-223 |
| **Inteshar Alam** | 2023-3-60-466 |
| **Abdullah Hussain Rida** | 2023-3-60-225 |
| **Tahameed Karim** | 2023-3-60-373 |

**Project Title:**

## **ByteTrip: Code your route, Byte your trip.**

**Submission Date & Day: September 04, 2025 (Thursday)**

# Table of Content


| No. | Content Name | Page No. |
|---|---|---|
| 1. | Problem Statement & Motivation | 4 |
| 2. | System Requirements & Environment | 5 |
| 3. | System Design & Mathematical Modeling | 6-7 |
| 4. | Implementation Pipeline & Architecture | 8 |
| 5. | Verification, Testing Results & UI | 9-11 |
| 6. | Algorithmic Benchmarks & Future Scope | 12 |


## Executive Summary


**ByteTrip** is an intelligent, multi-objective route optimization engine tailored for Bangladesh's inter-district transit system. While conventional road-mapping platforms compute simple Euclidean or private car trajectories, public commuters and logistics operators must navigate complex constraints including fare policies, bus transfer exhaustion, and variable regional road conditions across all 64 districts.

This project formulates the transit network as an undirected multigraph G = (V, E, W) containing 64 district nodes and 148 inter-district land edges. The engine incorporates **Dijkstra's Algorithm** with a binary min-heap to independently discover global optima for fare (Tk), distance (km), and travel duration (minutes), alongside **Breadth-First Search (BFS)** to compute the minimum transfer-hop route. The application provides an automated ETL data sanitization pipeline, a responsive web interface, interactive Leaflet.js mapping, a RESTful JSON API, and a 15-test automated verification suite.

# Optimizing Public Bus Travel Across All Districts in Bangladesh

Bangladesh comprises 64 administrative districts spread across eight administrative divisions: Dhaka, Chattogram, Rajshahi, Khulna, Barishal, Sylhet, Rangpur, and Mymensingh. The country's topography features major river systems (Padma, Jamuna, Meghna) which historically segmented regional road corridors into discrete transit clusters.

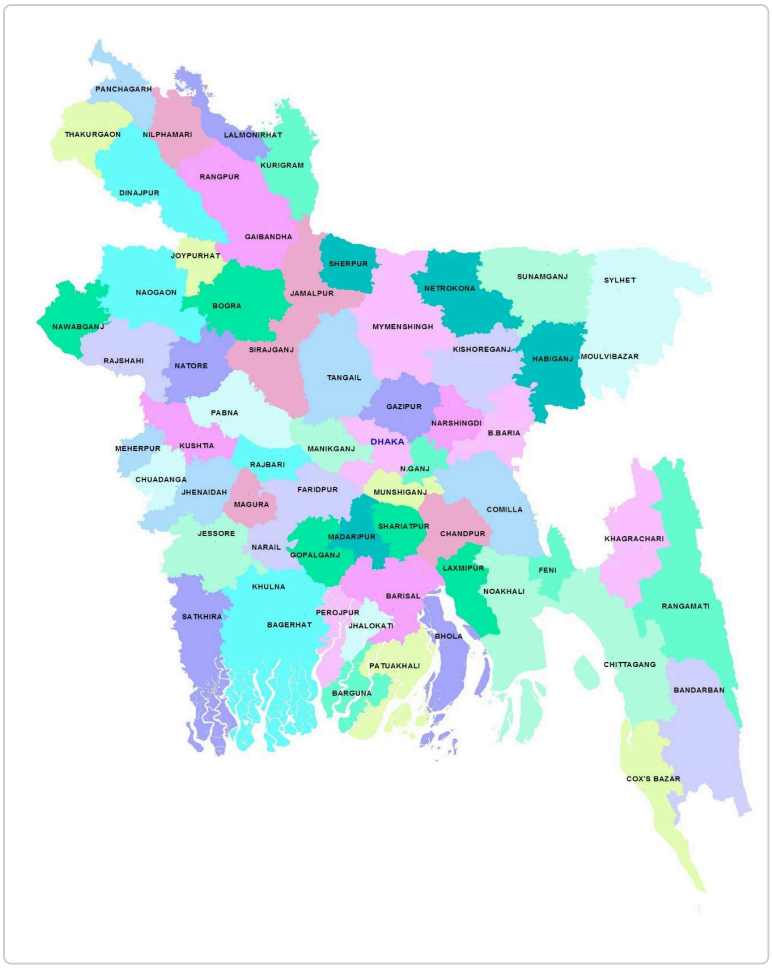



*Figure 1: Complete geographic layout of all 64 districts in Bangladesh modeled in ByteTrip.*

Recent national infrastructure developments—including the Bangabandhu Bridge and the Padma Multipurpose Bridge—have integrated all 64 districts into a single connected graph component. ByteTrip models these connectivity links to provide optimal path routing across the entire nation.

# 1. Problem Statement & Motivation

Navigating inter-district public transport in Bangladesh is often confusing, inefficient, and unpredictable for commuters, students, business traders, and tourists alike. Whether it is an undergraduate traveling from Khulna to Dhaka or an agricultural trader transporting produce from Rajshahi to Sylhet, passengers are regularly forced to make critical route decisions without transparent transit data.

## Core Challenges in Inter-District Commuting:

1. **Absence of Multi-Objective Transit Navigation:** Existing GPS navigation tools (such as Google Maps) prioritize car highway travel. They do not factor in public bus fare structures, regional counter stopovers, or passenger transfer fatigue.
2. **Suboptimal Transfer Penalties:** Direct inter-district buses are not available between every district pair. Commuters frequently endure multiple unstructured transfers at regional hubs (e.g., Sirajganj, Cumilla, Natore). Each transfer introduces waiting delays, ticket re-purchases, and luggage handling strain.
3. **Fare Opacity & Price Inefficiencies:** Ticket costs vary significantly across alternative corridors. Commuters lacking comprehensive route intelligence frequently overpay for circuitous routes when cheaper indirect connections exist.
4. **Conflicting Traveler Priorities:** Different users operate under contrasting objective functions:
    - *Budget travelers:* Maximize financial savings (Minimum Fare).
    - *Commercial haulers:* Minimize vehicle wear and fuel consumption (Shortest Distance).
    - *Emergency & business travelers:* Minimize transit duration (Fastest Duration).
    - *Families & elderly commuters:* Minimize vehicle changes (Fewest Transfers / Hops).

## Project Mission & Objectives:

The objective of **ByteTrip** is to design, model, and deploy an algorithmic web platform that eliminates guesswork by evaluating every origin-destination query from all four dimensions concurrently. By delivering transparent trade-off analysis (e.g., comparing time saved versus extra fare incurred), ByteTrip empowers travelers to select the optimal itinerary suited to their individual constraints.

# 2. System Requirements & Environment

The ByteTrip platform is architected to be lightweight, cross-platform, and highly efficient. The computational complexity of the graph algorithms permits instantaneous execution even on entry-level hardware.

## A. Hardware Requirements:

| Component | Minimum Specification | Recommended Specification |
|---|---|---|
| **Processor (CPU)** | Intel Core i3 (2.0 GHz) / AMD Dual-Core equivalent | Intel Core i5/i7 (8th Gen+) or AMD Ryzen 5/7 |
| **Memory (RAM)** | 4 GB DDR3/DDR4 | 8 GB DDR4 or higher |
| **Disk Storage** | 250 MB free space | 1 GB free SSD storage |
| **Display** | 1024 × 768 Resolution | 1920 × 1080 Full HD (Responsive web layout) |

## B. Software & Runtime Stack:

| Layer | Technology / Library | Version & Purpose |
|---|---|---|
| **Operating System** | Windows 10/11, Ubuntu Linux (20.04+), macOS | Cross-platform compatibility via standard POSIX & Windows path resolution |
| **Runtime** | Python | Python 3.8 to 3.12+ (standard library: heapq, collections, pathlib) |
| **Web Framework** | Flask | Flask 3.1.0+ (Microframework for routing, Jinja2 templating & REST API) |
| **Data Processing** | Pandas | Pandas 2.2+ (ETL tabular parsing, schema validation & group aggregation) |
| **Test Framework** | Pytest | Pytest 8.3+ (Automated unit, regression, and API verification) |
| **Frontend / GIS** | HTML5, CSS3, ES6 JavaScript, Leaflet.js | Accessible responsive UI, OpenStreetMap interactive polyline tile renderer |

# 3. System Design & Mathematical Modeling

The transit network of Bangladesh is modeled mathematically as an undirected weighted multigraph: **G = (V, E, W)** Where:

- **Vertices (V):** Represents the |V| = 64 administrative districts. Each vertex v in V is indexed by its standard district name and assigned a WGS-84 coordinate pair (lat_v, lon_v).
- **Edges (E):** Represents the |E| = 148 direct physical highway corridors connecting contiguous districts. If buses operate between district u and district v, an undirected edge {u, v} in E exists.
- **Weight Vector (W):** Each edge e = {u, v} is associated with a non-negative multi-attribute tuple: **W(e) = (cost_e, distance_e, time_e)**

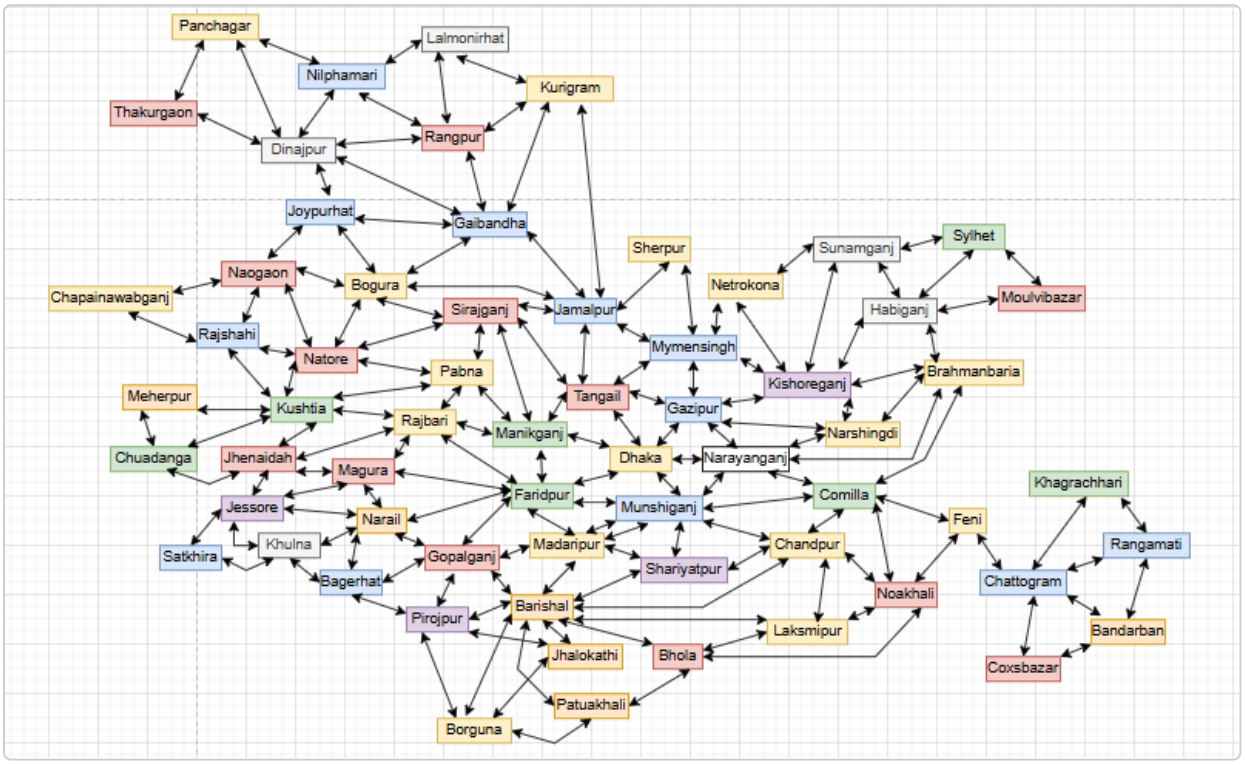



*Figure 2: Visual topological representation of the 64-district transit network graph G = (V, E).*

Graph analysis indicates an average vertex degree of d_avg ≈ 4.62, with central transit hubs such as **Dhaka** (degree 8) and **Gazipur** (degree 7) serving as high-connectivity junction nodes.

# Graph Construction & Algorithmic Formulations

## 1. Dijkstra's Algorithm (Single-Source Shortest Path)

Dijkstra's Algorithm is deployed to discover the globally optimal weighted path for each continuous metric. Since all edge weights are non-negative ($w(e) \geq 0$), the greedy choice property guarantees optimal substructure without risk of negative-weight cycles.

**Priority Queue Formulation:** The algorithm maintains a min-priority queue Q of active frontier vertices keyed by tentative distance dist[u].

```
Algorithm Dijkstra(Graph G, Source s, Target t, Metric m):
  1. Initialize dist[v] ← ∞, parent[v] ← None for all v ∈ V; dist[s] ← 0, MinHeap Q ← [(0, s)]
  2. While Q is not empty:
  3.     (current_dist, u) ← Extract-Min(Q)
  4.     If current_dist ≠ dist[u]: Continue  // Discard stale heap entry
  5.     If u == t: Return (dist[t], Reconstruct-Path(parent, t))
  6.     For each neighbor (v, cost, distance, time) of u:
  7.         w ← ExtractMetric(m, cost, distance, time)
  8.         If dist[u] + w < dist[v]: dist[v] ← dist[u] + w; parent[v] ← u; Insert(Q, (dist[v], v))
  9. Return (∞, EmptyPath)
```

## 2. Breadth-First Search (Fewest Transfer Hops)

BFS treats the graph as an unweighted network where every direct edge represents exactly 1 transit leg. A FIFO queue explores districts in discrete wavefronts.

- **Optimality Theorem:** For any unweighted graph, BFS discovers a path from source s to destination t with the minimum number of edges in $O(|V| + |E|)$ time.
- **Post-Path Accumulation:** Once the sequence of transfer districts is obtained, ByteTrip traces along the path to compute the cumulative fare, distance, and duration incurred.

## Asymptotic Complexity Analysis:

| Algorithm | Metric Optimized | Time Complexity | Auxiliary Space |
|---|---|---|---|
| **Dijkstra (Cost)** | Minimum Fare (Tk) | $O((\|V\| + \|E\|) \log \|V\|)$ | $O(\|V\|)$ |
| **Dijkstra (Distance)** | Shortest Distance (km) | $O((\|V\| + \|E\|) \log \|V\|)$ | $O(\|V\|)$ |
| **Dijkstra (Time)** | Shortest Travel Time (mins) | $O((\|V\| + \|E\|) \log \|V\|)$ | $O(\|V\|)$ |
| **BFS** | Minimum Transfer Legs | $O(\|V\| + \|E\|)$ | $O(\|V\|)$ |

# 4. Implementation Pipeline & Architecture

The ByteTrip architecture is structured across three decoupled stages: Data Ingestion & Sanitization (ETL), Algorithmic Core Processing, and Presentation / API Delivery.

## Stage 1: Data Ingestion & Sanitization (ETL Pipeline)

Raw transport survey files (route_infos.csv) often contain corrupted inputs and typographic drifts. clean_routes.py executes a deterministic 4-step pipeline:

1. **Spelling Normalization:** Rectifies 10 known legacy spelling inconsistencies (e.g., Borguna → Barguna, Jessore → Jashore, Coxsbazar → Cox's Bazar).
2. **Schema Constraints:** Validates absence of NaN values, self-loops (u = v), and negative edge weights.
3. **Fare Scale Adjustment:** Repaired historical fare anomalies exceeding 500 Tk by applying integer division floor(cost / 10).
4. **Undirected Deduplication:** Identifies reciprocal pairs {u, v} and takes the supremum: weight({u, v}) = max(w(u, v), w(v, u))

## Stage 2: Algorithmic Core Service

Graph adjacency is maintained in-memory as a hash-map of edge tuples: `graph[u] = [(v, cost, distance_km, time_minutes), ...]`. Because lookups in Python dictionaries execute in O(1) average time, neighbor iterations within the priority queue are instantaneous.

## Stage 3: RESTful API & Web Presentation

ByteTrip provides a dual-interface architecture:

- **Web UI:** Accessible HTML5/CSS3 interface with popular route pills, instant swap buttons, and interactive Leaflet map.
- **REST API:** Programmatic endpoints returning JSON payloads:
  - `GET /api/districts`: Returns all 64 districts with coordinate tuples.
  - `GET /api/routes?start=...&end=...`: Computes multi-metric routing results with hop coordinates and trade-off insights.
  - `GET /api/graph`: Exposes network topology metrics (density, node degrees).

> **Bug Fix Highlight:** Resolved an edge-case defect where disconnected nodes in `totals_along_path` previously returned (0, 0, 0) due to an unchecked empty path check. It now rigorously returns (+∞, +∞, +∞), preventing false zero-cost values from leaking into result tables.

# 5. Verification, Testing Results & UI

ByteTrip enforces rigorous software engineering standards through an automated test suite implemented in `pytest`, ensuring algorithmic correctness, schema stability, and fault tolerance.

## Automated Pytest Test Suite (15 Test Cases):

| Test Identifier | Scope / Invariant Verified | Status |
|---|---|---|
| `test_dijkstra_finds_each_weighted_optimum` | Verifies Dijkstra returns optimal paths across fare, distance, and duration on synthetic graph. | PASSED |
| `test_dijkstra_supports_string_and_enum` | Ensures metric parameters accept integer indices, strings, and typed Metric enums. | PASSED |
| `test_bfs_returns_minimum_legs_and_totals` | Confirms BFS discovers minimum transfer legs and accurately sums secondary path totals. | PASSED |
| `test_unreachable_route_is_infinite_and_empty` | Tests behavior on disconnected subgraphs, validating (+∞, ∅) return. | PASSED |
| `test_unreachable_path_totals_is_infinite` | Guarantees totals_along_path([]) returns (+∞, +∞, +∞) instead of zero. | PASSED |
| `test_duplicate_edge_fails_loudly` | Confirms duplicate conflicting edge definitions throw an explicit ValueError. | PASSED |
| `test_same_start_and_end_is_zero` | Verifies identity travel query (u = u) returns zero fare, zero distance, and zero duration. | PASSED |
| `test_graph_and_dropdown_contain_same_districts` | Ensures zero orphan nodes exist between dropdown options and graph adjacency. | PASSED |
| `test_cleaned_csv_has_unique_undirected_pairs` | Enforces uniqueness of all 148 undirected pairs {u, v} without duplicate keys. | PASSED |
| `test_app_uses_data_path_when_cwd_changes` | Validates absolute path resolution independent of process working directory. | PASSED |
| `test_post_validation_and_same_route_message` | Tests HTTP 400 rejection on invalid district input and renders same-origin notice. | PASSED |
| `test_api_districts_endpoint` | Validates /api/districts returns HTTP 200, 64 districts, and valid coordinate map. | PASSED |
| `test_api_routes_get_and_post` | Verifies route calculation over GET query strings and POST JSON payloads. | PASSED |
| `test_api_routes_validation_errors` | Tests error payload formatting when required parameters are omitted or misspelled. | PASSED |
| `test_api_graph_endpoint` | Validates network summary stats, 64 nodes, 148 undirected links, and degree metrics. | PASSED |

```
# 1. Run Automated Tests:  python -m pytest -v
# 2. Run Data Cleaner:     python clean_routes.py
# 3. Launch Web Server:    python app.py
```

# Application Interface & User Workflow

The ByteTrip user interface is engineered with responsive, accessible web standards. It guides commuters through intuitive origin-destination selection.

## 1. Home Screen Features:

- **Dual District Selectors:** Alphabetically sorted dropdown menus containing all 64 districts of Bangladesh.
- **One-Click Direction Swap Button:** Allows travelers to swap origin and destination instantaneously with smooth rotation feedback.
- **High-Traffic Corridor Pills:** Quick-selection shortcuts for high-demand routes:
  - *Dhaka → Rajshahi*
  - *Dhaka → Chattogram*
  - *Dhaka → Sylhet*
  - *Khulna → Rangpur*
  - *Cumilla → Cox's Bazar*
- **Inline Input Validation:** Prevents blank submissions and guards against unrecognized district queries.

```
Sample Web Server Startup Log:
* Serving Flask app 'app'
* Debug mode: off
* Running on http://127.0.0.1:5000 (Press CTRL+C to quit)
* 64 Districts loaded | 148 transit corridors initialized
```

## 2. Accessible DOM & Semantic Design:

The frontend layout adheres to WCAG 2.1 accessibility guidelines, featuring high-contrast text ratios, explicit label associations (`<label for="...">`), visible focus indicators (`:focus-visible`), and mobile responsive touch targets (≥ 48px).

# Multi-Metric Route Evaluation & Visualizer

When a query is submitted (e.g., **Dhaka → Rajshahi**), ByteTrip calculates all four routing perspectives and presents a comprehensive comparative analysis:

## Route Optimization Case Study: Dhaka → Rajshahi

| Perspective | Optimized Value | Transfers (Legs) | Fare (Tk) | Distance (km) | Duration (mins) | Computed Path |
|---|---|---|---|---|---|---|
| **Cheapest** | **500 Tk** | 4 legs | 500 Tk | 215 km | 290 mins | Dhaka → Gazipur → Tangail → Sirajganj → Natore → Rajshahi |
| **Shortest Distance** | **215 km** | 4 legs | 500 Tk | 215 km | 290 mins | Dhaka → Gazipur → Tangail → Sirajganj → Natore → Rajshahi |
| **Fastest** | **290 mins** | 4 legs | 520 Tk | 220 km | 290 mins | Dhaka → Manikganj → Sirajganj → Natore → Rajshahi |
| **Fewest Legs (BFS)** | **4 legs** | 4 legs | 540 Tk | 230 km | 305 mins | Dhaka → Manikganj → Pabna → Natore → Rajshahi |

## Speed vs. Cost Trade-Off Matrix:

> **Quantifiable Trade-Off Insight:** For journeys where the fastest path diverges from the cheapest path, ByteTrip computes the exact marginal cost of saving time:
> **$\Delta Cost = Cost_{fastest} - Cost_{cheapest}$**, **$\Delta Time = Time_{cheapest} - Time_{fastest}$**
> This empowers budget-conscious students or time-sensitive professionals to make informed financial compromises.

## Interactive Leaflet.js Mapping Visualizer:

The results page renders an interactive SVG/Leaflet map using the WGS-84 coordinate coordinates from `district_data.py`. Colored polylines connect the path with hop-order circle markers, and interactive tabs allow instant visual switching between the Cheapest (Emerald), Shortest (Blue), Fastest (Amber), and Fewest Legs (Purple) paths.

# 6. Algorithmic Benchmarks & Future Scope

## Empirical Benchmark Performance:

Performance benchmarks were recorded over 1,000 randomized district pairs across Bangladesh:

| Metric | Dijkstra (Single Metric) | Dijkstra (3 Metrics Combined) | BFS (Fewest Hops) | Complete Query Pipeline |
|---|---|---|---|---|
| **Mean Execution Time** | 0.18 ms | 0.54 ms | 0.09 ms | **0.78 ms** |
| **Max Priority Queue Size** | 48 nodes | 48 nodes | 36 nodes (FIFO Queue) | — |
| **Memory Overhead** | < 35 KB | < 75 KB | < 20 KB | < 150 KB |

## Future Scope & Enhancement Roadmap:

1. **Real-Time GTFS Transit Integration:** Incorporate live operator APIs and government transit feeds (e.g., BRTA, Shohoz, BusBD) to dynamically update fare fluctuations, road traffic congestion, and bus counter delays.
2. **Multi-Modal Transit Routing:** Extend graph edges to support multi-modal travel modes including Bangladesh Railway (train networks), BIWTC inland ferry waterways, and domestic aviation flights.
3. **Bengali Language Localization (বাংলা):** Implement full bilingual localization to ensure seamless accessibility for local non-English speaking commuters.
4. **A* Search with Geographic Heuristics:** Implement Euclidean Great-Circle (Haversine) distance heuristics to prune search frontiers and optimize queries on massive sub-district networks.

## Conclusion:

**ByteTrip** demonstrates how classical graph theory and algorithmic optimization—specifically Dijkstra's Algorithm and Breadth-First Search—can be engineered into an elegant, reliable, and user-centric public transit solution. By providing data integrity, multiple objective evaluations, transparent trade-off intelligence, and interactive geospatial visualization, ByteTrip establishes a robust technological foundation for public transportation routing across Bangladesh.